

Problem Solving using C



Copyright © www.infomaxacademy.com

Introduction to C



Programming language

A **programming language** is a formal computer **language** designed to communicate with machine, particularly a computer. **Programming languages** can be used to create programs to control the behaviour of a machine or to express algorithms.

➤ **Machine oriented languages or low level language**

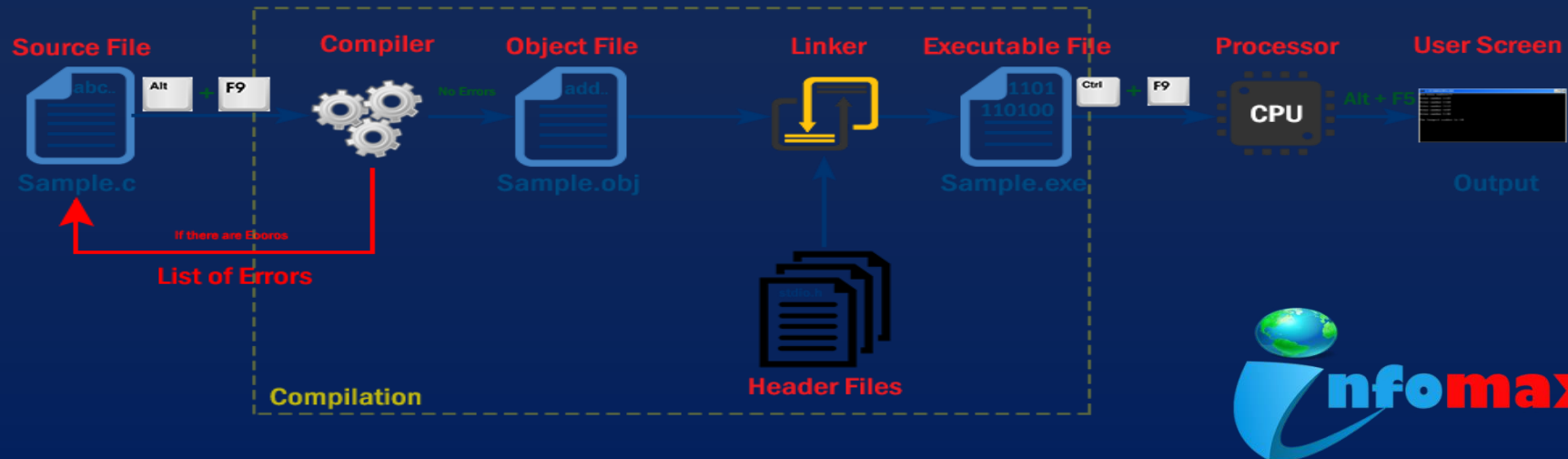
A low-level language is a programming language that provides little or no abstraction of programming concepts, and is very close to writing actual machine instructions. Two good examples of low-level languages are assembly and machine code.

➤ **Problem Oriented language or High level language**

A high-level language is a computer programming language that isn't limited by the computer, designed for a specific job, and is easier to understand. It is more like human language and less like machine language. However, for a computer to understand and run a program created with a high-level language, it must be compiled into machine language. Example: BASIC, C, C++, Cobol, FORTRAN, Java, Pascal, Perl, PHP, Python,

- **Compiler** : A compiler is system software which converts programming language code into binary format in single steps. In other words Compiler is a system software which can take input from other any programming language and convert it into lower level machine dependent language.
- **Interpreter** : It is system software which is used to convert programming language code into binary format in step by step process (line by line).
- **Assembler**: An assembler is system software which is used to convert the assembly language instruction into binary format in step by step process. An assembler is system software which is used to convert the assembly language instruction into binary format.

C is a remarkable language. Designed originally by Dennis Ritchie, working at AT&T Bell Laboratories in New Jersey, it has increased in use until now it may well be one of the most widely written computer languages in the world. C is a structured language. It allows variety of programs in small modules. It is easy for debugging, testing, and maintenance if a language is a structured one.



History of C

- The C programming language was designed in the early 1970s by Dennis M. Ritchie an employee from Bell Labs (AT&T)
- C is the result of a development process that started with an older language called BCPL.
- BCPL was developed by Martin Richards, and it influenced a language called B, which was invented by Ken Thompson.
- B led to development of C in the 1970.
- To alter this situation, ANSI established a committee in the beginning of 1983 to create a standard for C, which was implemented in 1987.

Structure of C program

Include header file section

Global declaration section

main()

{

Declaration part

Executable part

}

User-defined functions

{

Statements

}

Character set is a set of valid characters that a language can recognize.

Letters	Digits	White Space
Capital A to Z	All decimal digits 0 to 9	Blank Space
Small a to z		Horizontal tab
		Vertical tab
		New Line

Special Character

,	Comma	&	Ampersand
.	Dot	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus
'	Apostrophe	+	Plus
"	Quotation Mark	<	Less than
!	Exclamation mark	>	Greater than
	Vertical bar	()	Parenthesis
/	Slash	[]	Bracket
~	Tilde	%	Percent
_	Underscore	#	Hash
\$	Dollar	=	Equal to
?	Question mark	@	At the rate



C tokens are the basic building blocks in C language which are constructed together to write a C program. Each and every smallest individual units in a C program are known as C tokens.

- Delimiter
- Keywords
- Identifiers
- Constants
- Strings
- Special Symbols
- Operators

A **delimiter** is a sequence of one or more characters used to specify the boundary between separate, independent regions in plain text or other data streams

Delimiters	Use
: Colon	Useful for label
; Semicolon	Terminate the statement
() parenthesis	Used in expression and function
[] Square bracket	Use for array declaration
{ } Curly Brace	Scope of the statement
# Hash	Pre-processor directive
, Comma	Variable separator

Keywords are pre-defined words in a C compiler. Each keyword is meant to perform a specific function in a C program.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers are names of variables, functions, and arrays. They are user-defined names, consisting sequence of letters and digits, with the letter as the first character.

Rules for constructing identifier name in c:

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.

Constant values does not change during the execution of the program

Numerical constants:

- **Integer constants:** These are the sequence of numbers from 0 to 9 without decimal points or fractional part or any other symbols. It requires minimum two bytes and maximum four bytes.

Ex: 10, 20, + 30, - 14

- **Real constants:** It is also known as floating point constants.

Ex: 2.5, 5.342, -65.8

Character constants:

- **Single character constants:** A character constant is a single character. Characters are also represented with a single digit or a single special symbol or white space enclosed within a pair of single quote marks.

Eg: 'a', '8', '@'

- **String constants:** String constants are sequence of characters enclosed within double quote marks.

Eg: "122@ABC", "infomax", "444"

It is a data name used for storing a data value. Its value may be changed during the program execution. The value of variables keeps on changing during the execution of a program.

Declaration of Variable:

<data type> <variable name>

Ex: int num;

Initialization on variable:

<data type> <variable name>= <value>;

Ex: int num=34;

 char gen='m';

 float amt=444.5;

Data type is a special keyword used to allocate sufficient memory space for the data, in other words Data type is used for representing the data in main memory (RAM) of the computer.

Data type	Size (Bytes)	Range	Format Specifiers
Char	1	– 128 to 127	%c
Unsigned char	1	0 to 255	%c
Short or int	2	– 32,768 to 32, 767	%i or %d
Unsigned int	2	0 to 655355	%u
Float	4	3.4e – 38 to +3.4e +38	%f or %g
Long	4	2147483648 to 2147483647	%ld
Unsigned long	4	0 to 4294967295	%lu
Double	8	1.7e – 308 to 1.7e+308	%lf
Long double	10	3.4e – 4932 to 1.1e+4932	%lf

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators :

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Increment and Decrement Operators
- Conditional Operators
- Special Operators

Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction and multiplication on numerical values (constants and variables).

Operator	Description	Example (int a=10,b=7;)
+	Addition	$a+b = 17$
-	Subtraction	$a-b = 3$
*	Multiplication	$a*b = 70$
/	Division	$a/b = 1$
%	Modulus (Remainder)	$a\%b = 3$
Note: modulus operator only used with int data type		

- Write program a which accept two integer number from user finds its addition, subtraction ,multiplication and division.
- Write a program which accept length and breath (Floating point values) of rectangle and find the area of rectangle.
- Write a program which accept one integer an prints its square and cube.
- Write a program which accept the number of days and print it week and days format. For Example input is 10 days the result is 1 week and 3 days.
- Write a program which accept two time (Hour and Min.) and calculate the sum to two time. For Example:

Time 1	1 Hour	40 Min
Time 2	2 Hour	30 Min
Then Sum is	4 Hour	10 Min

Relational Operators

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Relational operators are used in decision making and loops.

Operator	Description	Example (int a=5,b=10)
>	greater than	$a > b = 0$
<	less than	$a < b = 1$
>=	greater than or equal to	$a \geq b = 0$
<=	less than or equal to	$a \leq b = 1$
==	equal to	$a == b = 0$
!=	not equal to	$a != b = 1$

Logical Operators

Logical operators are used to combine two or more condition for building complex expression.

Operator	Description	Example
&&	Logical AND. True only if all operands are true	If a = 1 and b = 0 then (a && b) is False
	Logical OR. True only if either one operand is true	If a = 1 and b = 0 then (a b) is True
!	Logical NOT. True only if the operand is 0	If a = 1 then !(a) is False

Bitwise Operators

Bitwise operators perform manipulations of data at bit level. These operators also perform shifting of bits from right to left. Bitwise operators are not applied to float or double.

Operator	Description	Example: a=0,b=1
&	Bitwise AND	$a \& b = 0$
	Bitwise OR	$a b = 1$
^	Bitwise exclusive OR	$a \wedge b = 1$
<<	Left Shift	
>>	Right Shift	

EX: Bitwise Operators in C

a = 0001000

b = 2

a << b = 0100000

Assignment Operators

- An assignment operator is used for assigning a value to a variable. The most common assignment operator is =
- It assigns the value from right to left side variable.

Operator	Description	Example
=	Simple Equal	$C = A + B$ will assign the value of $A + B$ to C
+=	Add & assignment	$C += A$ is equivalent to $C = C + A$
-=	Subtract & assignment	$C -= A$ is equivalent to $C = C - A$
*=	Multiply & assignment	$C *= A$ is equivalent to $C = C * A$
/=	Divide & assignment	$C /= A$ is equivalent to $C = C / A$
%=	Modulus & assignment	$C \% = A$ is equivalent to $C = C \% A$



- Write a program which accept two integer number and swap them values.
- Write a program which accept two integer number and swap them values without using any third variable.
- Write a program to accept a number and print n^2 , n^3 and n^4 .
- Write a program to input a single digit(n) and print a 3 digit number created as $\langle n(n+1)(n+2) \rangle$ e.g. if you input 7, then it should print 789. Assume that the input digit is in range 1-7.

Increment/Decrement Operator

Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one in C programs.

Type of Increment Operator

- Pre increment/decrement Operator (++a, --a)
- Post increment/decrement Operator (a++, a--)

In **pre increment / decrement** first increment/decrement the value of variable and then used inside the expression (initialize into another variable)

In **post-increment / decrement** first value of variable is used in the expression (initialize into another variable) and then increment / decrement the value of variable.

Conditional Operator

The conditional operator (? :) is a ternary operator. This operator needs 3 operands to perform the operation.

Syntax:

Variable = <test expression> ? <expression 1> : <expression 2>

Or

<test expression> ? <expression 1> : <expression 2>

The conditional operator works as follows:

- Test Expression is Condition
- Expression 1 is Statement Followed if Condition is True
- Expression 2 is Statement Followed if Condition is False

Precedence and Associativity of operators

- **Precedence:** If more than one operators are involved in an expression, C language has a predefined rule of priority for the operators. This rule of priority of operators is called operator precedence.
- **Associativity :** If two operators of same precedence (priority) is present in an expression, Associativity of operators indicate the order in which they execute.

Operator	Description	Associativity
() [] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of <i>type</i>) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Generally **Comments** are used to provide the description about the Logic written in program. Comments are not display on output screen.

When we are used the comments, then that specific part will be ignored by compiler.

In 'C' language two types of comments are possible

- Single line comments
- Multiple line comments

Single line comments

Single line comments can be provided by using / /.....

Multiple line comments

Multiple line comments can be provided by using /* */



printf is a predefined function in "**stdio.h**" header file, by using this function, we can print the data or user defined message or variable value on console or monitor.

Syntax

```
printf("user defined message");
```

Syntax

```
printf("Format specifiers",value1,value2,..);
```

Example of printf function

```
int a=10; double d=13.4;
```

```
printf("%f%d",d,a);
```

Input in C

scanf()

scanf() is a predefined function in "stdio.h" header file. It can be used to read the input value from the keyword.

Syntax

```
scanf("format specifiers",&value1,&value2,.....);
```

Example of scanf function

```
int a; float b; scanf("%d%f",&a,&b);
```

In the above syntax format specifier is a special character in the C language used to specify the data type of value.

Type casting is process to convert a variable from one data type to another data type. For example if we want to store a integer value in a float variable then we need to typecast integer into float.

There are tow type of conversion

- Implicit
- Explicit

Implicit: When the type conversion is performed automatically by the compiler without programmers intervention, such type of conversion is known as **implicit type conversion**

Explicit: The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as **type casting**.

Syntax: (data_type) var_name;

Example: int =65;

char ch=(char)a;

- Write a program which accept a character from user and print its ASCII value.
- Write a program which accept a ASCII value from user and print its corresponding character.
- Write a program which accept a alphabet from user in lower case and print in Upper case.
- Accept any 5 integer number from user and finds it average. (Numbers in integer and average in float).

Math Function

The **math.h** header defines various mathematical functions and one macro. All the functions available in this library take **double** as an argument and return **double** as the result.

- **double ceil(double x):** The C library function double ceil(double x) returns the smallest integer value greater than or equal to x.

syntax : double ceil(double x)

- **double floor(double x):** The C library function double floor(double x) returns the largest integer value less than or equal to x.

syntax : double floor(double x)

- **double fabs(double x):** The C library function double fabs(double x) returns the absolute value of x.

syntax : double fabs(double x)



Math Function Cont..

- **double fmod(double x, double y)** : The C library function double fmod(double x, double y) returns the remainder of x divided by y.
syntax : double fmod(double x, double y)
- **double sqrt(double x)**: The C library function double sqrt(double x) returns the square root of x.
syntax : double sqrt(double x)
- **double pow(double x, double y)**: The C library function double pow(double x, double y) returns x raised to the power of y i.e. x^y .
syntax : double pow(double x, double y)

Conditional Statements and Loop



The statements inside your source files are generally executed from top to bottom, in the order that they appear. *Control flow statements*, however, break up the flow of execution by employing decision making, looping, and branching, enabling your program to *conditionally* execute particular blocks of code.

- Conditional Statement
- Looping statement
- Jumping Statement

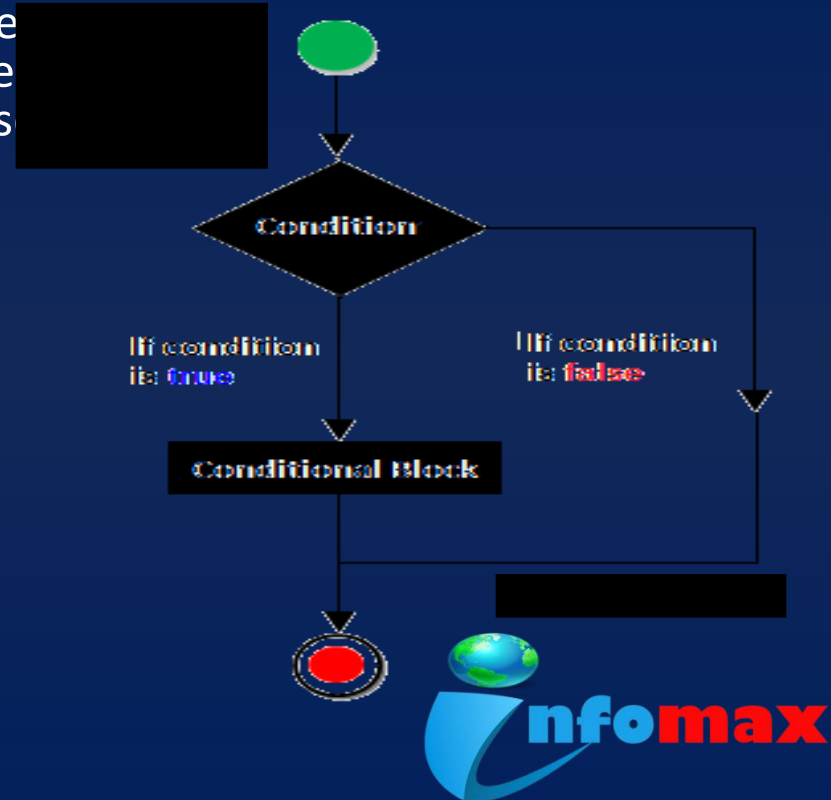
Decision making statement

- **Decision making statement** is depending on the condition block need to be executed or not which is decided by condition. In C programming language there are three types of conditional statement:
- If statement
- If else statement
 - Nested if else statement
 - Else if ladder statement
- Switch case statement

This is the simplest form of branching statement. When we have only one condition to evaluate, we should use this statement. It performs a course of action if the condition evaluates to true, otherwise, it ignores the action.

Syntax:

```
if(conditional expression)
{
    Statement 1
    Statement 2
    .....
    Statement n
}
```



Lab Assignment

- Write a program to check whether the entered number is positive or not.
- Write a program to input an integer number and print its absolute value.
- Write a program input any char from user and convert it in upper case if it is in lower case.

if-else statement

The if else control structure is used where either of two action are to be performed depending upon the result of a conditional expression. It contains two block of statement. If the conditional expression evaluates to true, the statement(s) in if block are executed and if the result is false, then statement(s) in the else block get executed.

Syntax:

```
if(conditional expression)
{
    statement 1
    statement 2
    .....
}
else
{
    statement 1
    statement 2
    .....
```

- Write a program to input an integer number and check number is **EVEN** or **ODD**.
- Write a program to input any integer number and check number is **BUZZ Number** or Not. (A number that ends with 7 or divisible by 7, is called buzz number).
- Write a program to display whether the entered character is Uppercase or Lowercase.
- Write a program to check whether the entered number is divisible by 5 or Not.
- Write a program input cost price, selling price and check whether is profit or loss and also print profit/loss amount.
- Write a program input any one digit integer number and its square contains as last digit or not. Ex: $5=25$, $6=36$

IF ELSE IF LADDER

The if-else-if ladder statement executes one condition from multiple statements. The execution starts from top and checked for each if condition. The statement of if block will be executed which evaluates to be true. If none of the if condition evaluates to be true then the last else block is evaluated.

```
if(condition_expression_One)
{
    statement1;
}
else if (condition_expression_Two)
{
    statement2;
}
else if (condition_expression_Three)
{
    statement3;
}
else
{
    statement4;
}
```

- Write a program to input serial number and print the corresponding day of week.
- Write a program input 5 subject marks and find total, percentage and grade based on given condition.

Grade A – per ≥ 80

Grade B – per ≥ 70 and < 80

Grade C – Per ≥ 60 and < 70

Grade D – per ≥ 50 and < 60

per < 50 is FAIL

- Create a program to calculate the amount that a customer pays for the taxi that he hires based on the following conditions.

KMS Travelled	Amount per KM(Rs)
First 10 Kms	30
Next 20 Kms	20
Next 40 Kms	15
Above 70 Kms	12

- Nested if is a conditional statement, which is used when more than one condition is required to be checked at the same time in a program. The statements are executed from the top to the bottom, checking each condition, whether it meets the criteria or not. If the condition is true, then it executes the block of associated statements of true part, else it goes to the next condition to execute.
- Nesting an if means you can use one if or else if statement inside another if or else if statement. The inner if(s) must always be terminated before the outer if(s).

Lab Assignment

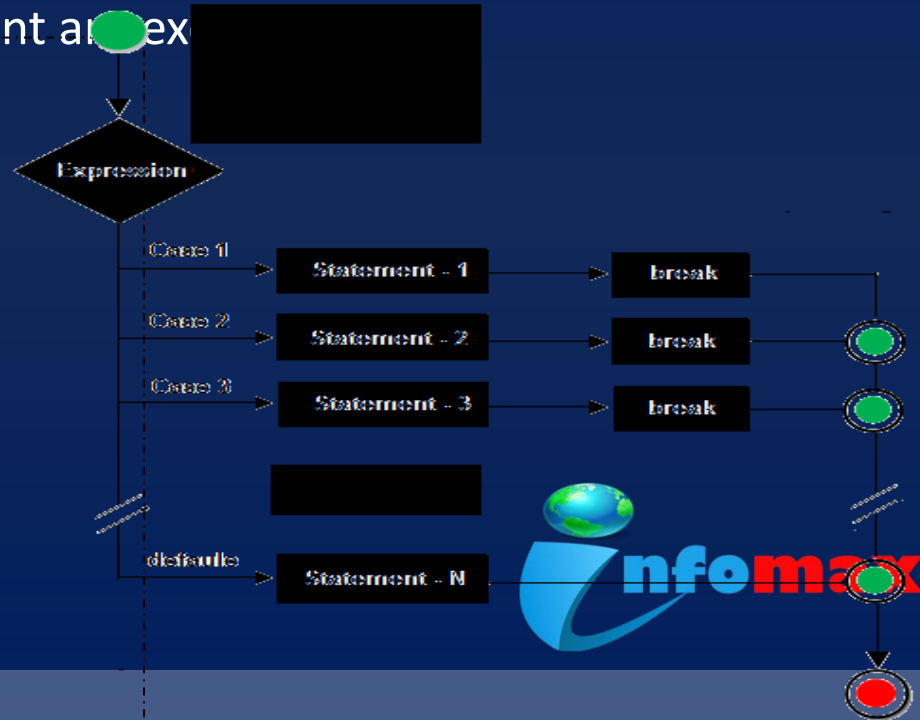
- Program that will accept three numbers from the user , and find the largest number.

Switch Statement

Switch is a multiple-branch selection statement, which successively tests the value of an expression against a list of **integer** or **character** constants (floating point expression, for example, are not allowed). When a match is found, the statements associated with that constant are executed.

Syntax:

```
switch(expression)
{
    case constant:
        statement;
        break;
    case constant:
        statement;
        break;
    ..
    ..
}
```



- The value of the **expression** is tested against the constants specified in the case statements in a top-down order..
- When a **match is found**, the statement sequence associated with that case is **executed** until the break statement or the end of the **switch** statement is reached.
- When **break** is encountered in a **switch**, program execution "**jumps**" to the line of code following the **switch** statement.
- The **default** statement is executed if no matches are found.
- The **default** is optional.
- The **switch** differs from the **if** in that **switch** can **only test for equality**, whereas **if** can evaluate any type of relational or logical expression.
- No two case constants in the same switch can have identical values

Lab Assignment



51



Looping statement

Looping statement are the statements execute one or more statement repeatedly several number of times. In C programming language there are three types of loops:

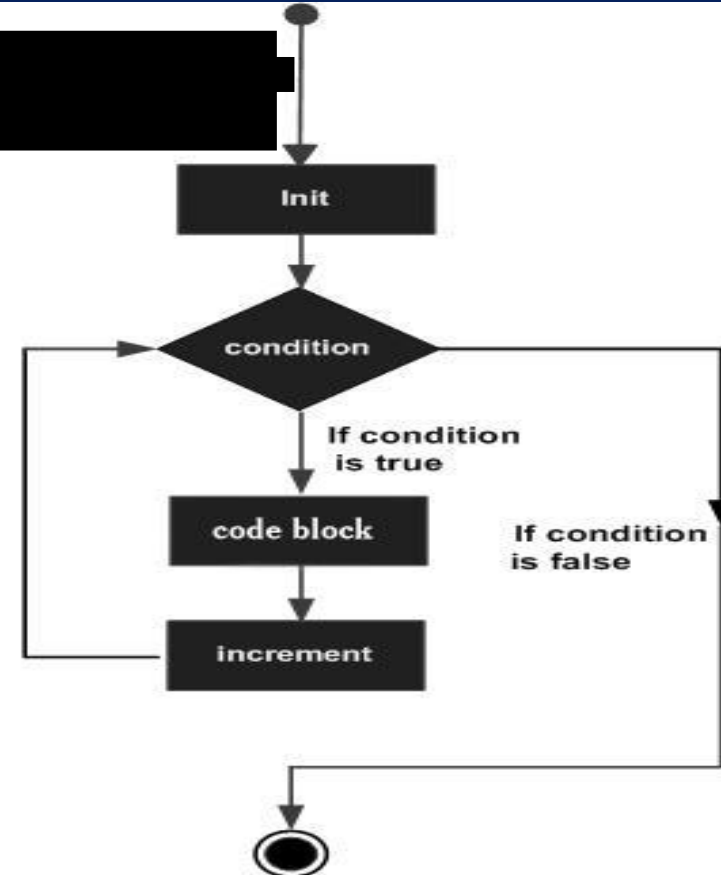
- For loop
- While loop
- Do while loop

For loop

for loop is a statement which allows code to be repeatedly executed. For loop contains 3 parts: Initialization, Condition and Increment or Decrement.

Syntax:

```
for(initial value ;condition; increment/decrement)
{
    statement 1
    statement 1
    .....
    statement n
}
```



1. Create a program to create a table of given number.
2. Write a program to print cube of numbers 1 to 5.
3. Create a program to print the factorial of a number. Factorial is $1*2*3...*n$.
4. Write a program to accept a number from the user and display its divisors.
Ex: 12= 1,2,3,4,6,12
5. Write a program which accept a number (n) and find the sum from 1 to n.
6. Write a program to input any 10 integers number and find largest number.
7. Create a program that will display the elements of Fibonacci series between 1 and 100. eg. 0,1,1,2,3,,5,8,13,21,34,55,89

Variation in FOR Loop

- A for loop can have multiple variations that makes more flexible and efficient.
 - Multiple initialization and update expression
 - Optional expression
 - Infinite loop
 - Empty loop

Multiple initialization and update expression

A for loop may contain multiple initialization and update expressions. The multiple expressions must be separated by commas.

Example:

Multiple Initialization

```
int i,j;
```

```
for(i=1,j=20; i=10; i++,j--)
```

Multiple update expression

```
{
```

```
    System.out.println(i+" "+j)
```

```
}
```


By now you are familiar that a for loop contains three types of expressions: initialization expression(condition) and an update expression. But, all these expressions are optional, i.e., we can skip any or all of these expressions depending upon the requirement of the program.

Case 1:

```
Int i=2;
for( ;i<=20;i=i+2)
{
    printf("%d",i);
}
```

Case 2:

```
Int i=2;
for( ;i<=20; )
{
    printf("%d",i);

    i=i+2;
}
```

Case 3:

```
Int i=2;
for( ; ; )
{
    printf("%d",i);
    i=i+2;
    if(i>20)
    {
        break;
    }
}
```



A for loop will execute its statement block infinite number of times if we skip the condition. If the test expression (condition) is not present in the loop, it will never return false. Hence, it will execute endlessly and become an infinite loop.

Case 1

```
int l  
for(i=1; i++)  
{  
    System.out.println(i);  
}
```

Case 2

```
int i;  
for( ; ; )  
{  
    System.out.println(i);  
}
```

Case 3

```
int i;  
for(i=1; i<=i; i++)  
{  
    System.out.println(i);  
}
```

Here , $i \leq i$ will always be true.



A loop without any statement in its body is called an empty loop

```
int i
for(i=2;i<=50;i++)
{
}
```

Or

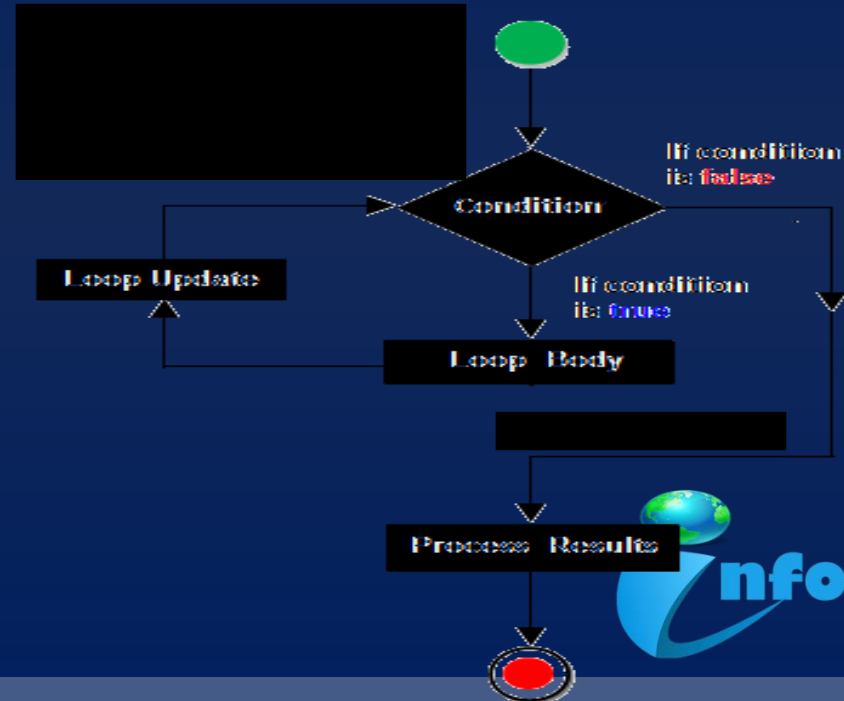
```
for(i=2;i<=50;i++);
```

While loop

The while loop is entry controlled loop. The while loop keeps on executing the block of statement till the specified test condition evaluates to true. When the test condition becomes false, the program control is transferred out of the loop and passes to the statement after the body of the loop.

Syntax:

```
While(condition)
{
    Statement 1;
    Statement 2;
    .....
    Statement n;
}
```



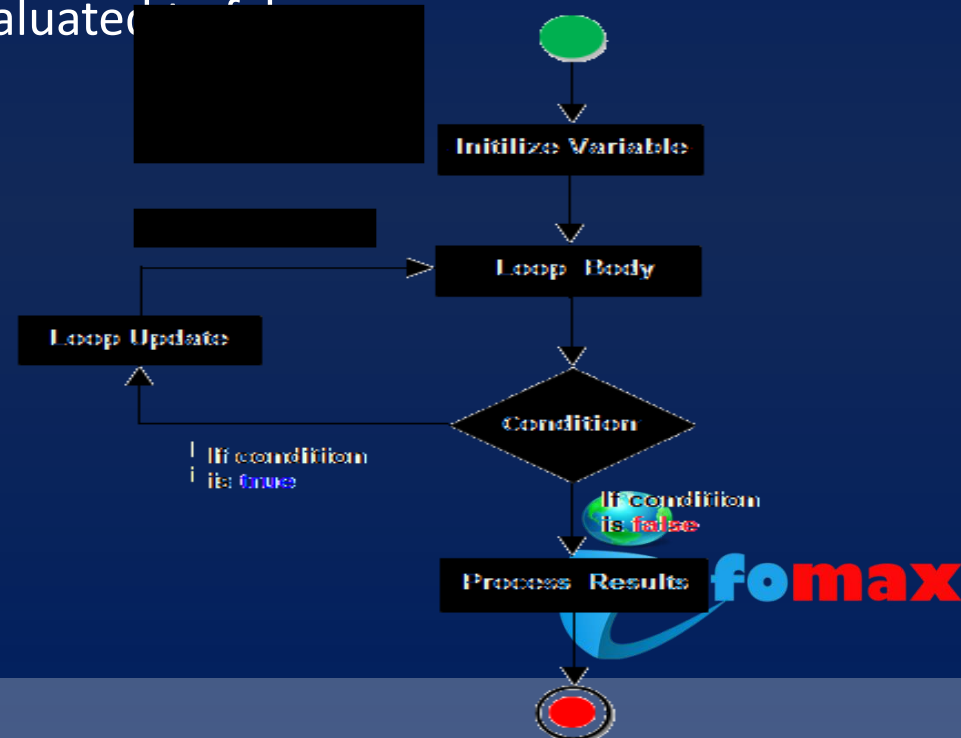
1. Write a program that will find the **reverse** of a given number.
2. Write a program which accept a number from user and **count the number of digits**.
3. Write a program that will find the **Sum of digits** of a given Number.
4. Write a program which accept a integer number from user and check, number is **Palindrome** or Not. Eg 121 ,444,100001 etc.
5. Write a program which accept a 3 digit integer number and check number is **Armstrong** or not. Ex 153, 370,371,407.
6. Write a program input any integer number and finds its largest and smallest digit. Ex. 92584 the min=2 and max=9
7. Write a Program which accept a number from user and check whether it is a **DISARIUM Number** or not. **Ex.** 135 is a DISARIUM ($1^1+3^2+5^3 = 135$), some other **DISARIUM** are 89, 175, 518 etc.)

do-while Loop

The do while loop is an **exit controlled loop**, since the body of loop is executed first and then the test condition is evaluated. If it evaluates to false, then the loop is terminated, otherwise it is repeated. This means do while loop always executes **at least once**, even if the condition is evaluated to false.

Syntax:

```
do
{
    statement 1;
    statement 1;
    .....
    statement 1;
}while(condition);
```



- Write a program which print sum of negative numbers, sum of positive odd numbers and positive even numbers from a list of numbers entered by the user. The list terminates when the user enters a Zero.
- Write a program which accept integers number from user until the sum of all given number is less than 100.
-

Jumping Statement

C language provides us multiple statements through which we can transfer the control anywhere in the program.

There are basically 3 Jumping statements:

1. break jumping statements.
2. continue jumping statements.
3. goto jumping statements.
4. return Statement

Break Statements

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the end of any immediate loop or switch body.
- To do all this we have to specify a break jumping statements whenever we want to terminate from the loop.
- **Syntax:** break;
- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

NOTE: As break jumping statements ends/terminate loop of one level . so it is referred to use return or goto jumping statements , during more deeply nested loops.

Continue Statements

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the beginning of any immediate loop.
- To do all this we have to specify a continue jumping statements whenever we want to terminate any particular condition and restart/continue our execution.
- **Syntax:** continue;
- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

Goto Statements

- By using this jumping statements we can transfer the control from current location to anywhere in the program.
- To do all this we have to specify a label with goto and the control will transfer to the location where the label is specified.

Syntax:

```
<label>:  
    statement  
goto <label>;
```

Note :

- It is good programming style to use the break, continue and return instead of goto.
- However, the break may execute from single loop and goto executes from more deeper loops.

return statement

- return statement is used to transfer program's control from called function to calling function, it's secondary task is to carry value from called function to calling function.
- If function's return type is void, you can use return only to return program's control to the calling function, and if there is a return type you will have to return same type of value to the calling function.

Syntax:

return;

Or

return (value)

Array



An array is a collection of variable of the same data type that is referred a common name. Each element in array is associated with an index number and occupies contiguous memory location.

Characteristics of an array:

- It is a collection of similar (homogeneous) data type of element.
- The array index number always starts with 0(zero) so the last index is size-1.
- It always stored in contiguous memory location.
- The size and dimension of array is fixed.
- It is a linear data structure that stores the element in a sequence.

Declaration of an array:

`<Data_type> <array_name>[size];`

Example:

```
int num[5];
```

Initialization of an array:

```
int num[5]={4,5,6,5,4,3};
```

0	1	2	3	4
5	6	8	9	4

Num

String in C

Strings are actually one-dimensional array of characters terminated by a **null** character '\0'.

Declaration :

```
char <array_name>[size];
```

Example:

```
char name[20];
```

Initialization :

```
char greeting[] = "Hello";
```

OR,

```
char greeting[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Input output String:

gets(): Input String

puts(): Output/Print String

Functions	Description
<u>strcat ()</u>	strcat (str2, str1); – str1 is concatenated at the end of str2.
<u>strcpy ()</u>	strcpy (str1, str2) – It copies contents of str2 into str1.
<u>strlen ()</u>	int x=strlen(str1) it counts the number of characters in str1
<u>strcmp ()</u>	int x=strcmp(str1,str2) : Returns 0 if str1 is same as str2. Returns <0 if str1 < str2. Returns >0 if str1 > str2
<u>strlwr ()</u>	strlwr(str1) Converts string to lowercase
<u>strupr ()</u>	strupr(str1) Converts string to uppercase
<u>strrev ()</u>	strrev(str1) Reverses the given string



Functions



- A function is a block containing a number of statements, which perform a specific and well defined task(s). Every function has a unique name which is used to identify it in a program. Once a task is packaged in a function, it can be called from any where in a program through other statements.

Advantages of function:

- Function reduce the **complexity** of program by dividing the into smaller modules, thus making them easier to read and debug.
- Function enhance **reusability** of code and can be called from any where in a program and any number of time.
- Function are useful in hiding the implementation details.
- It is easier to edit a program, which is written using methods. While working on one function, one can focus on just that part of the program by debugging, constructing and making it perfect.

➤ Function can be broadly classified into two categories:

➤ **Built-in function or library functions**

These are inbuilt function, which are grouped together and stored in a standard library (Header Files). You can make use of these functions to get a predefined output, instead of writing your own code.

➤ **User defined Function :**

These function are designed by the user to carry out some specific tasks. These name of the function is given by the user according to his/her requirement; hence, it is called a user defined function.

Defining a Function

Syntax:

```
return_type function_name( parameter list )  
{  
    body of the function  
}
```

Return Type: The **return_type** is the data type of the value the function returns , if function does not return value the it will be void.

Function Name – This is the actual name of the function.

Parameters : The parameter list refers to the type, order, and number of the parameters of a function.

Function Body – The function body contains a collection of statements that define what the function does.



Categories of Function

Based on arguments and return type function are categorized into four category as follows:

- Method with no argument and no return value
- Method with argument and no return value
- Method with argument and with return value
- Method with no argument and with return value

Arguments to method/Function

- We can pass two type of arguments to a function
- We can pass any primitive data type argument to a c function, like int, char, float, long etc. This method of passing arguments is known as passing arguments by value.
- We can also pass reference data types, like address of variable or array.
- A method can be called in two different ways:
 - **Call by Value or Pass by Value**
 - **Call by Reference or Pass by Reference**

- **Call By Value:** When you pass parameters to a function by value, the method copies the value of actual parameter into the formal parameter. It creates its own copy of argument values and then uses in the method.
- **Call By Reference :** In this technique, actual arguments are passed by reference rather than passing by values. When function call is made, the reference of actual parameters is copied into formal parameters. Further this reference is used to access the values stored in a function. Since the function receive the location of the original value, the same value is used for all operations. If any changes ate made in the formal parameters, the original value automatically gets modified. This happen because the program has only one copy of the value.

- Recursion is the ability of a C function to call itself. Recursion is capable of defining an object in terms of a simpler case of itself. "Recursion is a function, which keeps on calling itself. So, when will the function stop? There should be if condition upon which recursion stops.

Example

```
/*non-recursive*/
```

```
int fact(int n)
```

```
{
```

```
    int t, answer;
```

```
    answer=1;
```

```
    for(t=1;t<=n;t++)
```

```
        answer=answer*t;
```

```
    return answer;
```

```
}
```

```
/*recursive*/
```

```
Int factr(int n)
```

```
{
```

```
    int answer;
```

```
    if(n==1)
```

```
        return 1;
```

```
    /*recursive call*/
```

```
    answer=factr(n-1)*n;
```

```
    return answer ;
```

```
}
```

Storage Classes



Structures and Unions



Structure is a collection of variables of different types under a single name.

Structure Definition in C

Keyword struct is used for creating a structure.

Syntax of structure

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ..
    data_type memeber;
};
```

Structure variable declaration:

```
struct <stucture_name> <var_name>;
```

Example:

```
struct person
{
    char name[50];
    int citNo;
    float salary;
};
```

```
Void main()
{
    struct person person1;
```

```
.....
}
```

Pointers



Pointer

- A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address. The general form of a pointer variable declaration is –

```
data_type * var_name;
```

Ex:

```
int *ptr;
```

```
int a =23,*ptr;
```

```
ptr=&a;
```

then

value of '*ptr' is 23

Application of Pointer

- Normal variable stores the value whereas pointer Pointers are more efficient in handling Arrays and Structures.
- Pointers allow references to function and thereby helps in passing of function as arguments to other functions.
- It reduces length of the program and its execution time as well.
- It allows C language to support Dynamic Memory management.

File Processing



A **file** represents a sequence of bytes on the disk where a group of related data is stored. File is created for permanent storage of data. It is a ready made structure.

In C language, we use a structure **pointer of file type** to declare a file.

C provides a number of functions that helps to perform basic file operations. Following are the functions,

Function	description
fopen()	create a new file or open a existing file
fclose()	closes a file
getc()	reads a character from a file
putc()	writes a character to a file
fscanf()	reads a set of data from a file
fprintf()	writes a set of data to a file
fseek()	set the position to desire point
ftell()	gives current position in the file

Opening a File or Creating a File

The `fopen()` function is used to create a new file or to open an existing file.

Syntax :

```
*fp = FILE *fopen(const char *filename, const char *mode);
```

mode	description
r	opens a text file in reading mode
w	opens or create a text file in writing mode.
a	opens a text file in append mode
r+	opens a text file in both reading and writing mode
w+	opens a text file in both reading and writing mode
a+	opens a text file in both reading and writing mode

➤ Closing a File

The `fclose()` function is used to close an already opened file.

Syntax :

```
int fclose( FILE *fp );
```

Input/Output operation on File

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char ch;
    fp = fopen("one.txt", "w");
    printf("Enter data");
    while( (ch = getchar()) != EOF)
    {
        putc(ch,fp);
    }
    fclose(fp);
    fp = fopen("one.txt", "r");

    while( (ch = getc(fp) != EOF)
    {
        printf("%c",ch);
    }
    fclose(fp);
    getch();
}
```

```
#include<stdio.h>
#include<conio.h>
struct emp
{
    char name[10];
    int age;
};
void main()
{
    struct emp e;
    FILE *p,*q;
    p = fopen("one.txt", "a");
    q = fopen("one.txt", "r");
    printf("Enter Name and Age");
    scanf("%s %d", e.name, &e.age);
    fprintf(p,"%s %d", e.name, e.age);
    fclose(p);
    do
    {
        fscanf(q,"%s %d", e.name, e.age);
        printf("%s %d", e.name, e.age);
    }
    while( !feof(q) );
    getch();
}
```